i4r-audit — team cheat sheet

How to set up, reserve a paper, run a full audit, and file the result. Everything in a framed box is a verbatim capture from a real run. Your audit's exact gate set is stamped into its run.json as `gates-<hash>`.

1. What you need (one-time setup, ~30 minutes)

#	Thing	Where / how	Why
1	**Claude Code** with an account that can run **Opus 5**	`npm install -g @anthropic-ai/claude-code`, then `claude` and log in	The coordinator and the five auditors run here
2	**ChatGPT desktop app**, logged in, workspace **with credits**	chatgpt.com/download — the Codex CLI ships inside the app at `/Applications/ChatGPT.app/Contents/Resources/codex`	The four adversarial critics run on GPT 5.6 Terra through it. **No credits = no critique rounds**; the skill detects this and stops before wasting a round
3	**R 4.4** with the notebook packages	`install.packages(c("haven","fixest","psych","rmarkdown"))`	The verification notebook re-derives every number in the comment
4	**Python 3** with the readers	`pip3 install pyreadstat pandas scipy numpy statsmodels jsonschema openpyxl`	Auditors cross-read every data format two ways
5	**pandoc** and **LibreOffice**	`brew install pandoc` · libreoffice.org	Builds the `.docx` and measures the real page count against the 10-page cap
6	The shared **Dropbox discipline folder**, synced locally	ask the coordinator for the invite	Papers, packages, reservations, and completed reports all live in one tree
7	The **skill** itself	next section	

2. Install the skill and prove it works

```
unzip i4r-audit.zip && cp -R i4r-audit ~/.claude/skills/
chmod +x ~/.claude/skills/i4r-audit/scripts/*.sh
bash ~/.claude/skills/i4r-audit/scripts/make_claims_script.sh --selftest
bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh --selftest
```

Expected — both lines, verbatim:

```
SELFTEST OK: 12 checks - R fallback, Stata-primary naming, author-inline, R-embedded, severity,
SELFTEST OK: 8 checks - claim, contention, idempotence, foreign release, conflict copy, complete
```

If either prints `FAIL`, stop and ask in the team channel before running anything.

3. The three iron rules

Never contact the paper's authors or the journal. All correspondence goes through I4R — no exceptions, not even a "quick clarifying question."

If you cannot verify a RETRACTION- or CORRIGENDUM-level claim after honest study, return the paper and take another. Do not skip it; do not let the AI decide for you.

Never name the discipline in any material a replicator will see. Everything is written to be reused verbatim for the next cohort.

4. Step 0 — reserve your paper (every time, before anything else)

Six people work this tree at once. The reservation is what stops two of you burning a day on the same paper.

```
export I4R_OWNER=yourname@yourmachine     # once per shell; defaults to user@host
bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh claim \
  "<discipline-root>/<Journal>/<Paper folder>"
```

What you will see — success, and what a collision looks like:

```
RESERVED: Example Paper Title (2024)  ->  .../reservations/Journal_of_Example_Studies_Example_Pa

REFUSED: reserved by ana@macbook since 2026-08-24T20:06:32Z
```

`REFUSED` means pick another paper — that is the system working, not an error. Useful extras:

```
bash .../reserve_paper.sh list "<discipline-root>"        # who holds what; 7-day-old claims sho
bash .../reserve_paper.sh release "<paper-dir>"           # give a paper back (yours only)
```

The claim waits ~15 s after writing so Dropbox can surface a same-moment competing claim as a "conflicted copy"; if that happens, the **earlier timestamp wins automatically** and the later claimant is told to pick another paper. Your reservation is released for you when the finished audit is filed.

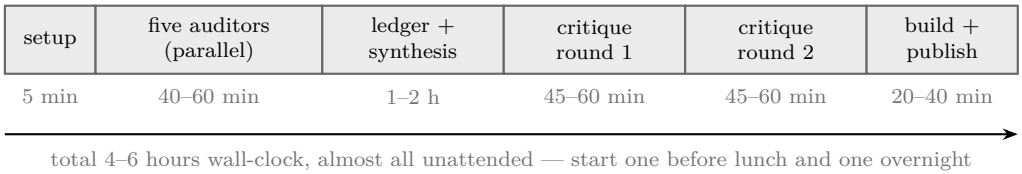
5. Run the audit

Open a terminal **in the paper's folder** and start Claude Code with Opus 5 selected. Then paste, adjusting the two paths:

```
/i4r-audit
Audit this paper. The article PDF is ./paper.pdf and the replication package is ./package.zip.
Run the whole pipeline to completion without stopping for approval: reserve nothing (I already
hold the reservation), set up the workspace, run all five auditors, synthesize with a ledger,
run the critique rounds to the production default, build the deliverable, generate the claims
script, and file the result with publish_completed.sh.
```

That is the whole job description. The skill does the rest and narrates as it goes.

What happens, and how long it takes



You mostly watch. The two moments that need a human: the coordinator may surface an adjudication it re-ran (read it — that is the audit's core), and any **gate FAIL** stops the pipeline with an explanation of what to fix.

What lands where when it finishes

```
<Paper folder>/
  audit/                        the full working record (reports, ledger, adjudication,
                                critique rounds, verification notebook, run.json, comment.json)
  I4R_comment_<Author>_<Year>.pdf     the deliverable, also copied to comments/
<discipline-root>/
  Completed reports/<Journal>/<Paper>/    exactly 4 files: PDF, claims script, comment.json, run
  reservations/                           your claim disappears from here automatically
```

The finalize step ends with lines like these — all `ok:` means the record is filed:

```
  ok: run.json validates against run.schema.json
  ok: comment.json validates against comment.schema.json
  ok: the notebook knitted with every claim reproducing
  ok: filed into .../registry/<audit-id> (index now holds N audits)
```

6. When something goes wrong

You see	It means	Do this
`FATAL: the Codex workspace is OUT OF CREDITS`	The critic provider has no balance; the probe caught it **before** burning a round	Refill (workspace owner), re-run the same round command
`FATAL: the critic model '...' is rejected`	The ChatGPT app updated and changed its default model — it did this twice in one week	Nothing to fix locally: the skill pins the model. Report it in the channel
`[fail] critic_x attempt N: exit 1, no output file` repeatedly	Look in `critique/round_N/log_*_try*.txt` — the real error is at the bottom	Usually credits or network; the log tells you
`REFUSED: reserved by ...`	Someone holds the paper	Pick another; check `reserve_paper.sh list`
`conflicted copy` mentioned by a claim	Two claims raced through Dropbox	The output already says who won (earliest timestamp); the loser picks another paper
Gate `FAIL: abstract is N words` / findings-table-in-body / table-vs-tags mismatch	The comment drifted from the format (a comment is a paper: Abstract 250 words on page 1, findings table in Appendix A)	The skill fixes and rebuilds; if you edited by hand, undo and let it
`WARNING: no PDF page count`	LibreOffice missing, so the 10-page cap is unverified	Install LibreOffice (item 5) and rebuild
R notebook exit nonzero	A claim genuinely failed to reproduce — this is a finding, not a bug	Read `verification_summary.csv`; do not "fix" the check to pass

7. Costs, so nobody is surprised

Observed across the calibration audits: roughly **2–3 M tokens per audit** on the Claude side, critique rounds of 45–60 minutes each on the GPT side, and 4–6 hours wall-clock end to end. The machine is busy but you are not: reserve in the morning, launch, check after lunch, file before dinner. `run.json` records your exact software versions and per-phase costs automatically — you never fill a form.